

Software Requirements Specification (SRS)

ApuntesUCT

Ecosistema Colaborativo de Material Académico Validado

Equipo de Análisis de Requisitos

Agosto de 2026

Índice

1. Introducción	3
1.1. Propósito	3
1.2. Alcance	3
1.3. Contexto del problema	4
1.4. Definiciones, acrónimos y abreviaciones	4
1.5. Referencias	5
2. Descripción general	6
2.1. Perspectiva del sistema	6
2.2. Funciones del sistema (resumen)	9
2.3. Clases de usuarios	10
2.4. Restricciones	10
2.5. Supuestos y dependencias	11
3. Stakeholders, actores y casos de uso	11
3.1. Stakeholders (lista)	11
3.2. Actores del sistema	12
3.3. Modelo de Casos de Uso	12
4. Requisitos específicos	16
4.1. Requisitos funcionales (RF)	16
4.2. Requisitos no funcionales (RNF)	17
4.3. Reglas de negocio (RB)	18
4.4. Interfaces externas	18

5. Casos de uso detallados	19
5.1. UC-01 Registrar e iniciar sesión	19
5.2. UC-02 Subir y clasificar material académico	19
5.3. UC-03 Moderar y verificar material	20
5.4. UC-04 Buscar y filtrar material	21
5.5. UC-05 Previsualizar, descargar y guardar favorito	21
5.6. UC-06 Evaluar o reportar material	22
6. Criterios de aceptación (extracto)	23
7. Trazabilidad (extracto)	23
8. Anexos	24

1. Introducción

1.1. Propósito

Este documento especifica los requisitos del sistema **ApuntesUCT**, una plataforma académica orientada a centralizar, clasificar, validar y distribuir material de estudio producido o compartido por estudiantes de la Universidad Católica de Temuco.

El objetivo general del proyecto es desarrollar e implementar un ecosistema tecnológico centralizado, accesible mediante una aplicación web y una aplicación móvil, orientado a la gestión, validación colaborativa y distribución hiper-contextualizada de material académico, con el fin de optimizar el tiempo de estudio y mejorar la confiabilidad de la información utilizada por los estudiantes.

El documento está dirigido a estudiantes del equipo de desarrollo, docentes evaluadores, stakeholders académicos, desarrolladores backend, desarrolladores web, desarrolladores mobile y responsables de control de calidad. Su finalidad es establecer una base común para delimitar alcance, requisitos, reglas de negocio, responsabilidades técnicas y criterios de aceptación del MVP.

1.2. Alcance

ApuntesUCT permitirá compartir, descubrir, evaluar, reportar, verificar y descargar material de estudio asociado a una estructura académica contextualizada. El sistema no se limita a almacenar archivos: incorpora moderación previa a la publicación, reputación de usuarios, reportes de calidad, favoritos, versionado básico y posicionamiento de resultados.

El MVP contempla:

- Autenticación de usuarios mediante correos institucionales.
- Catálogo académico de universidad, carrera, asignatura y profesor.
- Subida y clasificación de material académico.
- Flujo de moderación inicial en estado `PENDING_REVIEW`.
- Búsqueda contextual y filtros.
- Visualización, previsualización y descarga de documentos.
- Evaluaciones de 1 a 5 estrellas.
- Favoritos.

- Reportes de material incorrecto, duplicado, inapropiado o desactualizado.
- Verificación de material revisado.
- Reputación calculada desde backend.
- Posicionamiento de resultados usando señales de calidad.
- Versionado básico de materiales.

El proyecto será desarrollado de manera incremental durante aproximadamente cuatro sprints. Las funcionalidades complementarias, como un editor avanzado Markdown/LaTeX, no forman parte del núcleo obligatorio del MVP y solo se incorporarán si las funcionalidades principales se encuentran estables.

1.3. Contexto del problema

En la situación actual, los estudiantes de la Universidad Católica de Temuco dependen de canales informales para encontrar material de estudio: grupos de mensajería, correos, carpetas compartidas y repositorios no estructurados. Estos medios permiten circulación rápida, pero no garantizan orden, trazabilidad, vigencia ni calidad.

La falta de una herramienta centralizada genera tres problemas principales. Primero, el material queda disperso y es difícil recuperarlo cuando se necesita preparar una evaluación específica. Segundo, no existe un mecanismo confiable para distinguir apuntes vigentes de documentos obsoletos o con errores conceptuales. Tercero, el esfuerzo de estudiantes que producen material de calidad se pierde entre generaciones, porque no existe un sistema de reputación o reconocimiento que preserve y destaque esos aportes.

ApuntesUCT busca resolver esta situación mediante una plataforma especializada en el contexto académico UCT. La pregunta que intenta responder no es solamente “dónde encuentro un documento sobre una materia”, sino “cómo accedo al mejor material verificado para una asignatura, profesor, año y tipo de evaluación específicos”.

1.4. Definiciones, acrónimos y abreviaciones

MVP	Producto Mínimo Viable del sistema.
SRS	Software Requirements Specification o especificación de requisitos de software.
UCT	Universidad Católica de Temuco.

INT2	Equipo responsable de Web, backend e infraestructura.
INT4	Equipo responsable de la aplicación móvil Flutter.
API Gateway	Punto de entrada único que expone la API pública para Web y Mobile.
Microservicio	Servicio backend con responsabilidad de negocio propia y datos bajo su propiedad lógica.
Material	Recurso académico lógico subido por un usuario, con metadatos y una o más versiones.
MaterialVersion	Versión física o histórica de un material.
Moderación	Proceso de revisión previo o posterior a la publicación de un material.
Verificación	Marca que indica que un material cumplió el proceso de revisión definido por la plataforma.
Reputación	Puntuación calculada por backend a partir de aportes, evaluaciones y señales de calidad.
MinIO	Servicio de almacenamiento de archivos compatible con S3.
OpenAPI/Swagger	Especificación y documentación de la API pública.

1.5. Referencias

- IEEE 29148 como referencia conceptual para la estructura de especificación de requisitos.
- Documento de sistematización del proyecto ApuntesUCT.
- Especificación técnica de referencia del proyecto ApuntesUCT.
- Especificación de requisitos y tecnologías seleccionadas para ApuntesUCT.

2. Descripción general

2.1. Perspectiva del sistema

ApuntesUCT será una plataforma compuesta por dos clientes principales: una aplicación web responsiva y una aplicación móvil nativa. Ambos clientes consumirán la misma API pública expuesta por un API Gateway desarrollado en NestJS. La lógica de negocio residirá en el backend, distribuida en microservicios con responsabilidades separadas.

La arquitectura general considera:

- **Cliente Web:** interfaz responsiva desarrollada con React 19, Vite, TypeScript, Tailwind CSS, TanStack Query y React Router.
- **Cliente Mobile:** aplicación móvil desarrollada con Flutter 3.x y Dart 3.x, usando Riverpod, Dio, go_router, Freezed y json_serializable.
- **API Gateway:** entrada única para clientes Web y Mobile, responsable de enrutamiento, validación inicial, autenticación transversal y documentación pública.
- **Microservicios backend:** Auth, Catalog, Material, Quality y Search.
- **Persistencia:** PostgreSQL 17 con Prisma ORM 6.x, usando esquemas separados por servicio.
- **Archivos:** almacenamiento de documentos en MinIO, evitando guardar archivos completos en PostgreSQL.

Diagrama de arquitectura de software. La Figura 1 muestra la arquitectura lógica y de despliegue del MVP, considerando clientes, API Gateway, microservicios, red interna, PostgreSQL y MinIO.

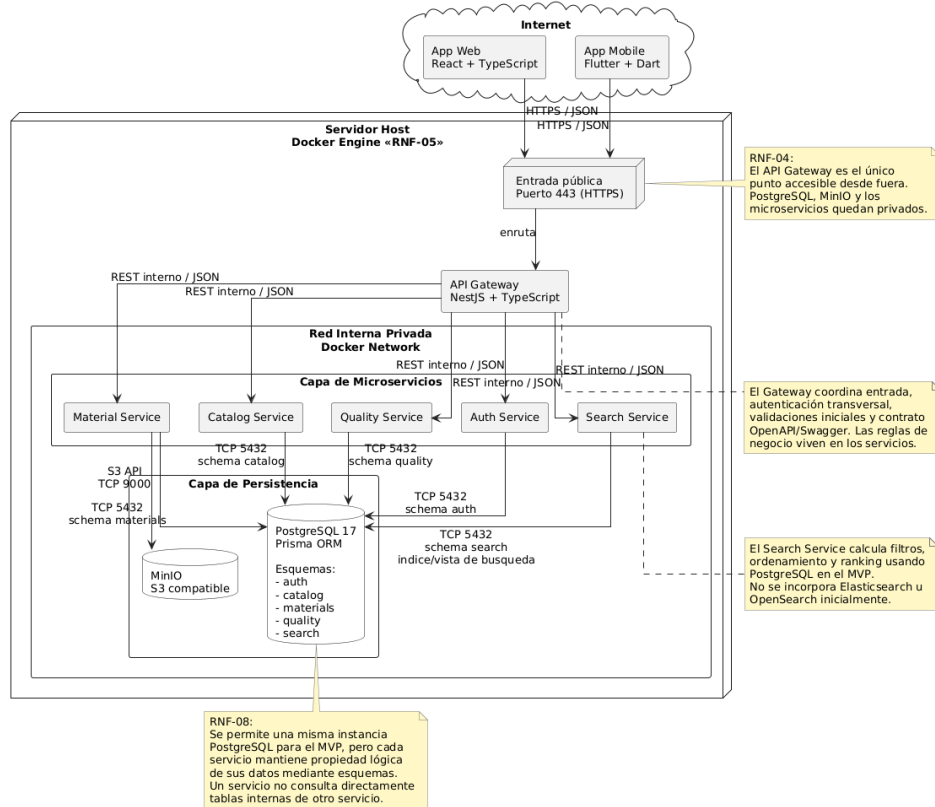


Figura 1: ApuntesUCT - Diagrama de Arquitectura de Software.

Diagrama de componentes UML. La Figura 2 muestra la separación de componentes principales del sistema, sus dependencias y las responsabilidades de cada bloque de software.

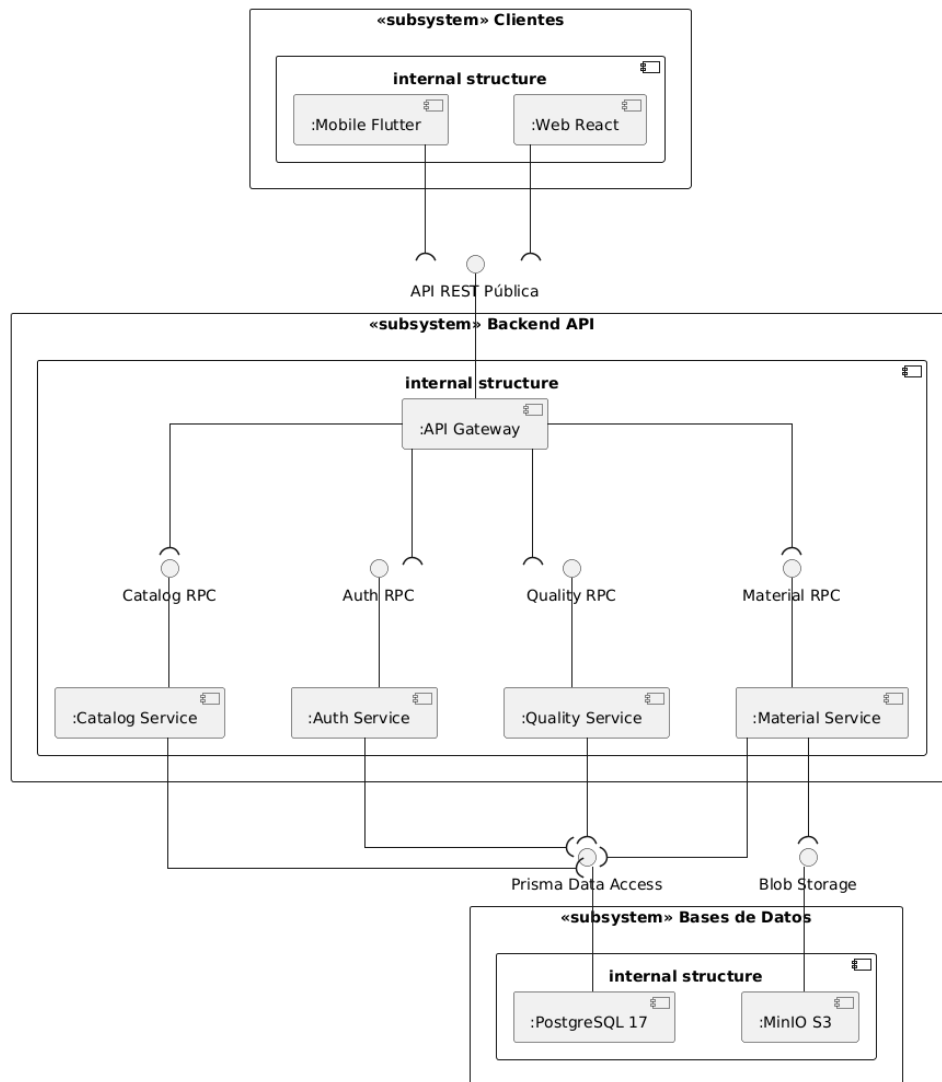


Figura 2: ApuntesUCT - Diagrama de Componentes UML.

Diagrama de modelo entidad-relación. La Figura 3 representa el modelo de datos del sistema, incluyendo las entidades principales del catálogo académico, usuarios, materiales, versiones, evaluaciones, favoritos, reportes y moderación.

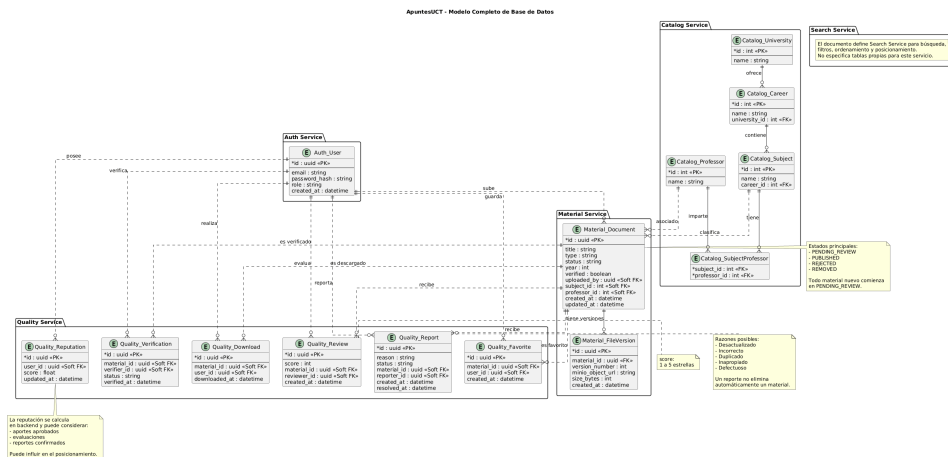


Figura 3: ApuntesUCT - Diagrama de Modelo Entidad-Relación (MER).

2.2. Funciones del sistema (resumen)

- Registrar usuarios y autenticar sesiones.
- Gestionar usuarios, roles y permisos.
- Mantener un catálogo académico de universidad, carrera, asignatura y profesor.
- Permitir la subida de material académico multiformato.
- Clasificar cada material mediante universidad, carrera, asignatura, profesor, año y tipo de material.
- Mantener todo material nuevo en cuarentena hasta su revisión.
- Buscar y filtrar material con criterios académicos.
- Previsualizar documentos compatibles, especialmente PDF.
- Descargar material publicado y autorizado.
- Calificar documentos mediante estrellas.
- Reportar contenido desactualizado, incorrecto, duplicado o inapropiado.
- Guardar y consultar favoritos.
- Administrar moderación, verificación y retiro de contenido.

- Calcular reputación de usuarios desde backend.
- Ordenar resultados usando señales de relevancia, valoración, verificación, actualidad, reputación y reportes.

2.3. Clases de usuarios

Estudiante registrado

Usuario autenticado que busca, visualiza, descarga, evalúa, reporta y guarda material como favorito.

Estudiante aportante

Estudiante registrado que sube material académico y consulta el estado de sus aportes.

Moderador

Usuario autorizado para revisar material pendiente, confirmar reportes y aplicar decisiones de calidad.

Administrador

Usuario con permisos de gestión sobre catálogo, usuarios, moderación, verificación y retiro de material.

Equipo INT2

Equipo responsable de implementar Web, API Gateway, microservicios, persistencia, seguridad, API y despliegue.

Equipo INT4

Equipo responsable de implementar la aplicación móvil y consumir la API pública definida por INT2.

2.4. Restricciones

- El backend del MVP debe implementarse con Node.js 22 LTS, TypeScript 5.x y NestJS 11.
- No se utilizará FastAPI/Python como backend del MVP.
- La aplicación web no será HTML Vanilla; se desarrollará con React.
- La aplicación móvil será desarrollada con Flutter.
- Web y Mobile deben consumir la misma API pública.
- Los clientes no deben acceder directamente a PostgreSQL, Prisma, MinIO ni microservicios internos.
- El API Gateway no debe alojar reglas de negocio centrales.

- Para el MVP se usará REST/HTTPS/JSON. No se incorporará RabbitMQ ni Kafka inicialmente.
- La búsqueda inicial se implementará sobre PostgreSQL. No se incorporará Elasticsearch/OpenSearch en el MVP.
- Los documentos locales tendrán inicialmente un límite de 15 MB.
- La arquitectura debe ser realizable dentro de cuatro sprints.

2.5. Supuestos y dependencias

- Los estudiantes disponen de conexión a internet para utilizar la plataforma.
- La universidad, carreras, asignaturas y profesores podrán representarse mediante un catálogo académico estructurado.
- El equipo INT2 entregará un contrato OpenAPI/Swagger suficiente para la integración mobile.
- Los servicios backend podrán ejecutarse localmente mediante Docker Compose.
- PostgreSQL y MinIO estarán disponibles en el entorno local y en el entorno de demostración.
- Los archivos pesados o videos podrán manejarse mediante enlaces externos seguros cuando excedan el alcance de almacenamiento local.
- La moderación inicial será realizada por usuarios autorizados antes de publicar material.

3. Stakeholders, actores y casos de uso

3.1. Stakeholders (lista)

Stakeholder	Interés / Necesidad
Estudiantes UCT	Encontrar material de estudio útil, vigente y específico para sus asignaturas, profesores y evaluaciones.

Estudiantes aportantes	Compartir apuntes de calidad, conservar sus aportes y recibir reconocimiento mediante reputación.
Moderadores	Revisar material, resolver reportes y mantener estándares mínimos de calidad.
Administradores	Gestionar catálogo, usuarios, permisos, verificación y control general del sistema.
Docentes y unidades académicas	Reducir circulación de material erróneo u obsoleto y promover mejores prácticas de estudio.
Equipo INT2	Implementar backend, web, infraestructura, seguridad, reglas de negocio y contrato API.
Equipo INT4	Implementar aplicación móvil, experiencia de usuario mobile e integración con API pública.
Docentes evaluadores del proyecto	Evaluar alcance, coherencia técnica, trazabilidad de requisitos y cumplimiento del MVP.

3.2. Actores del sistema

- **A1 Estudiante registrado** (humano).
- **A2 Estudiante aportante** (humano, especialización de estudiante registrado).
- **A3 Moderador** (humano).
- **A4 Administrador** (humano).
- **A5 Cliente Web** (sistema cliente).
- **A6 Cliente Mobile** (sistema cliente).
- **A7 MinIO** (sistema de almacenamiento de archivos).

3.3. Modelo de Casos de Uso

El modelo de casos de uso UML de ApuntesUCT se divide en tres vistas funcionales para mantener legibilidad: autenticación, búsqueda/catálogo y material/calidad. En conjunto, estas vistas delimitan las acciones esenciales del MVP: autenticación, búsqueda, subida, moderación, evaluación, reportes, favoritos, descarga y administración.

La Figura 4 muestra los casos de uso relacionados con registro, inicio y cierre de sesión, recuperación de acceso, edición de perfil y suspensión de cuenta.

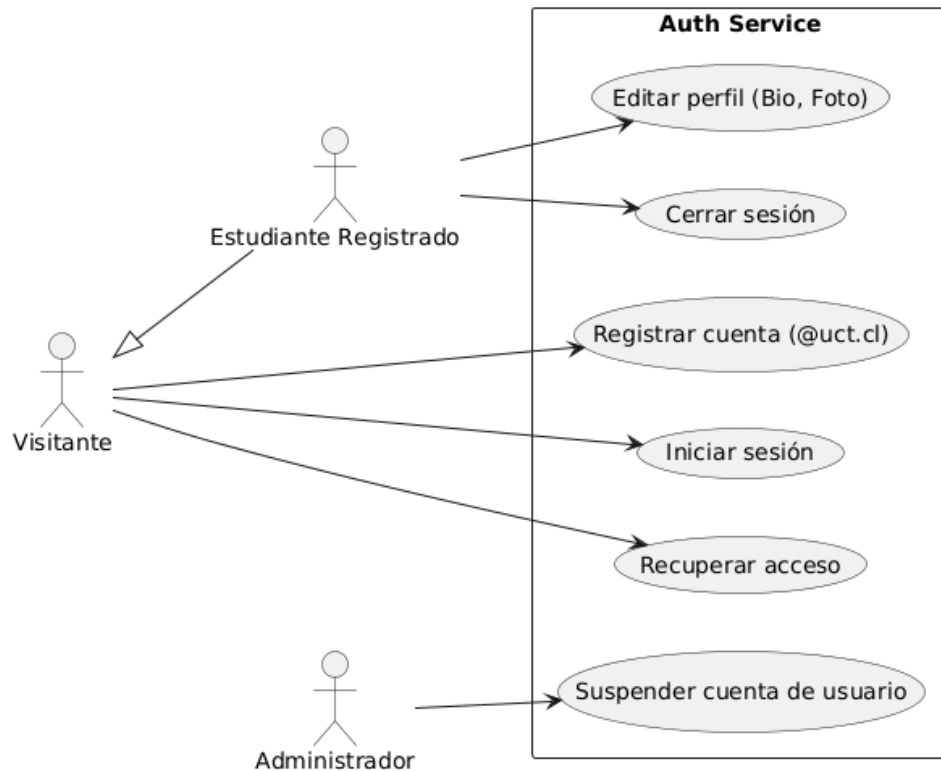


Figura 4: ApuntesUCT - Diagrama de Casos de Uso UML - Auth.

La Figura 5 muestra los casos de uso relacionados con búsqueda de material, filtros, ordenamiento, paginación, recomendaciones y administración del catálogo.

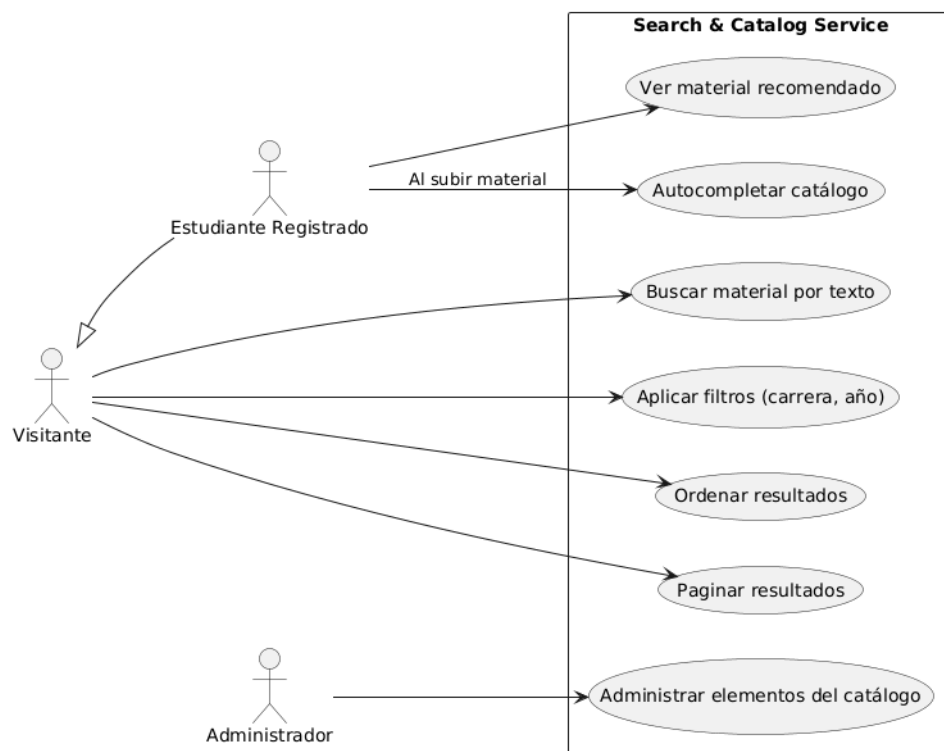
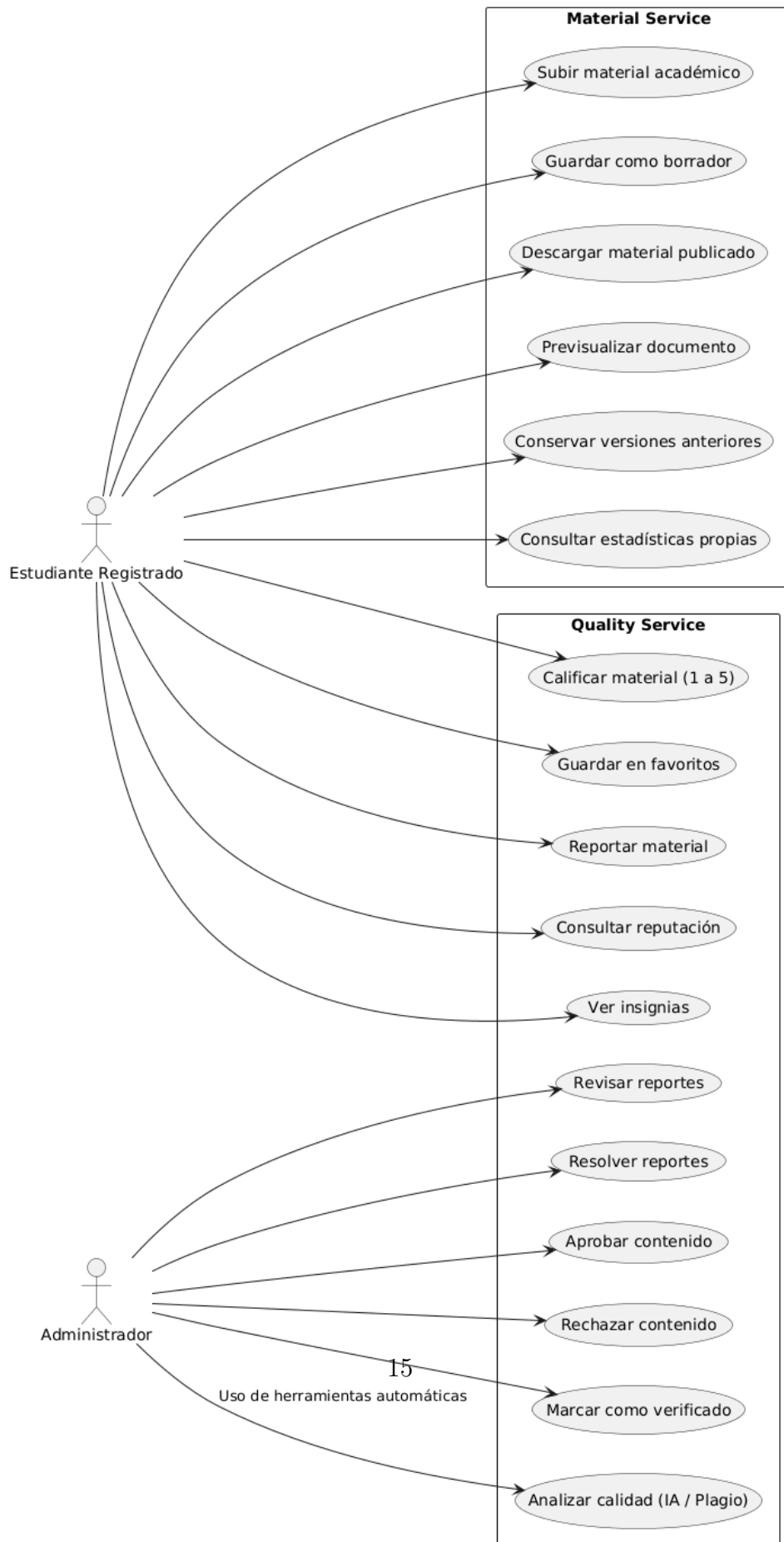


Figura 5: ApuntesUCT - Diagrama de Casos de Uso UML - Search/Catalog.

La Figura 6 muestra los casos de uso relacionados con materiales académicos, descargas, previsualización, versiones, evaluaciones, favoritos, reportes, reputación, insignias y moderación de contenido.



4. Requisitos específicos

4.1. Requisitos funcionales (RF)

RF-01: Gestión de identidad y autenticación. El sistema debe permitir registro, inicio de sesión, cierre de sesión y recuperación de acceso cuando corresponda. El registro inicial estará restringido a correos institucionales @uct.cl o @alu.uct.cl. (UC-01)

RF-02: Clasificación y subida multiformato. El sistema debe permitir subir material académico y asociarlo a universidad, carrera, asignatura, profesor cuando corresponda, año y tipo de material. Los documentos locales tendrán inicialmente un límite de 15 MB. Los videos o archivos pesados podrán utilizar enlaces externos. (UC-02)

RF-03: Flujo de moderación. Todo material nuevo debe ingresar en estado PENDING_REVIEW y ser revisado antes de quedar publicado. (UC-03)

RF-04: Búsqueda contextual y filtros. El sistema debe permitir buscar material mediante texto y filtros por carrera, asignatura, profesor, año, tipo de material, valoración y otros criterios disponibles. (UC-04)

RF-05: Previsualización multimedia. El sistema debe permitir visualizar documentos compatibles, especialmente PDF, sin obligar al usuario a descargar el archivo previamente. Los enlaces multimedia compatibles podrán mostrarse mediante contenido embebido seguro. (UC-05)

RF-06: Sistema colaborativo de evaluación y reputación. El sistema debe permitir calificar material de 1 a 5 estrellas y utilizar evaluaciones junto con otras acciones relevantes para calcular reputación desde backend. (UC-06)

RF-07: Módulo de reportes. El sistema debe permitir reportar material desactualizado, incorrecto, duplicado, inapropiado o defectuoso. Los reportes deben quedar registrados y pasar por revisión. (UC-06)

RF-08: Favoritos. El sistema debe permitir a un usuario guardar y consultar materiales favoritos. (UC-06)

RF-09: Versionado básico. El sistema debe conservar versiones anteriores de un material cuando exista una nueva versión. (UC-02)

RF-10: Verificación. El sistema debe permitir que un material sea marcado como verificado cuando cumple el proceso de revisión definido por la plataforma. (UC-03)

RF-11: Posicionamiento. El sistema debe ordenar los resultados utilizando señales como relevancia, valoración, verificación, actualidad, reputación, reportes y uso. (UC-04)

RF-12: Administración. El sistema debe permitir a usuarios autorizados revisar materiales, resolver reportes, aprobar o rechazar contenido, retirar material y administrar elementos del catálogo según sus permisos. (UC-03)

4.2. Requisitos no funcionales (RNF)

RNF-01: Arquitectura y acoplamiento. El sistema debe utilizar una arquitectura de microservicios con un API Gateway. Web y Mobile deben consumir la misma API pública y la lógica de negocio no debe residir en los clientes.

RNF-02: Rendimiento. Las consultas habituales, especialmente búsqueda y carga de resultados, deberían responder aproximadamente en 2 segundos o menos bajo condiciones normales de red e infraestructura. Este valor constituye un objetivo práctico del MVP.

RNF-03: Compatibilidad y experiencia de usuario. La web debe ser responsiva y la aplicación móvil debe ejecutarse en plataformas soportadas por Flutter, adaptándose a los tamaños de pantalla definidos para el MVP.

RNF-04: Seguridad de datos. El sistema debe validar entradas, sanitizar datos y enlaces externos, validar formato y tamaño de archivos, aplicar autenticación y autorización en backend, proteger credenciales y evitar exposición directa de PostgreSQL o MinIO.

RNF-05: Despliegue y portabilidad. Los componentes backend deben poder ejecutarse mediante Docker y Docker Compose. El proyecto deberá contar con un entorno de hosting para la demostración final.

RNF-06: Documentación de API. La API pública debe documentarse mediante OpenAPI/Swagger, incluyendo endpoints, métodos, parámetros, headers, autenticación, requests, responses, códigos HTTP, errores, filtros y paginación cuando corresponda.

RNF-07: Mantenibilidad. Cada microservicio debe mantener separación entre Presentation, Application, Domain e Infrastructure. Las reglas de negocio importantes deben estar separadas de controladores, ORM y detalles de infraestructura.

RNF-08: Independencia de datos. Cada microservicio debe ser responsable de sus propios datos. Para el MVP se permite una única instancia de PostgreSQL con esquemas separados por servicio, pero no se permite consultar directamente tablas internas de otro servicio.

RNF-09: Integración entre plataformas. Web y Mobile deben consumir la misma API pública. Los cambios necesarios para Mobile deben resolverse mediante coordinación con INT2 y evolución documentada del contrato API.

RNF-10: Pruebas. El backend deberá contar con tests para autenticación, creación de material, moderación, búsqueda, evaluaciones, reportes y favoritos. Web y Mobile deberán cubrir componentes críticos de sus respectivas interfaces.

4.3. Reglas de negocio (RB)

- RB-01:** Todo usuario debe iniciar sesión para subir material.
- RB-02:** Los usuarios registrados pueden descargar material publicado.
- RB-03:** Los usuarios pueden evaluar y reportar material publicado.
- RB-04:** Los administradores y moderadores autorizados pueden revisar y moderar contenido.
- RB-05:** Todo material debe registrar, cuando corresponda, universidad, carrera, asignatura, profesor, año y tipo de material.
- RB-06:** Todo material nuevo comienza en estado `PENDING_REVIEW`.
- RB-07:** Un material rechazado no aparece en las búsquedas normales.
- RB-08:** Un material retirado no aparece en las búsquedas normales.
- RB-09:** Un reporte no elimina automáticamente un material.
- RB-10:** Los reportes deben registrarse y revisarse antes de aplicar una sanción o retiro.
- RB-11:** Un material con reportes confirmados puede disminuir su visibilidad o volver a revisión.
- RB-12:** La reputación se calcula en backend y puede considerar aportes aprobados, evaluaciones recibidas, reportes confirmados y comportamiento relevante.
- RB-13:** La reputación puede influir en el posicionamiento, pero no determina por sí sola que un material sea válido.
- RB-14:** La etiqueta de material verificado indica cumplimiento del proceso de revisión, no garantía absoluta de corrección académica.
- RB-15:** Una nueva versión de material no debe destruir necesariamente la versión anterior.

4.4. Interfaces externas

- API pública REST/HTTPS/JSON:** Web y Mobile consumirán la misma API expuesta por el API Gateway. El cliente no debe conocer qué microservicio implementa cada endpoint.
- OpenAPI/Swagger:** La API pública debe contar con documentación de contrato para coordinación entre INT2 e INT4.
- MinIO:** El Material Service utilizará MinIO para almacenar archivos. Los clientes no accederán directamente a rutas internas del almacenamiento.
- PostgreSQL:** Los microservicios persistirán datos estructurados en PostgreSQL mediante Prisma ORM, manteniendo esquemas separados por servicio.
- Enlaces multimedia externos:** Videos o archivos pesados podrán almacenarse como enlaces externos seguros cuando no corresponda guardarlos directamente en MinIO.

5. Casos de uso detallados

5.1. UC-01 Registrar e iniciar sesión

Actor principal: Estudiante registrado.

Precondiciones: El usuario posee correo institucional válido @uct.cl o @alu.uct.cl.

Postcondiciones: El usuario queda registrado o autenticado, y puede acceder a las funcionalidades permitidas por su rol.

Flujo principal

1. El usuario ingresa a la aplicación Web o Mobile.
2. El usuario selecciona registro o inicio de sesión.
3. El sistema solicita credenciales y valida el formato del correo institucional.
4. El API Gateway recibe la solicitud y la deriva al Auth Service.
5. El Auth Service valida credenciales, rol y estado de la cuenta.
6. El sistema entrega una sesión o token válido para consumir la API pública.

Flujos alternativos

A1: Correo no institucional. Si el correo no pertenece a los dominios permitidos, el sistema rechaza el registro.

A2: Credenciales incorrectas. Si las credenciales no son válidas, el sistema informa el error sin iniciar sesión.

5.2. UC-02 Subir y clasificar material académico

Actor principal: Estudiante aportante.

Precondiciones: El usuario está autenticado y el catálogo académico contiene los datos necesarios para clasificar el material.

Postcondiciones: El material queda registrado con sus metadatos, archivo o enlace asociado, y estado PENDING_REVIEW.

Flujo principal

1. El usuario selecciona la opción de subir material.
2. El sistema solicita título, descripción, universidad, carrera, asignatura, profesor cuando corresponda, año y tipo de material.
3. El usuario adjunta un documento o ingresa un enlace externo permitido.

4. El sistema valida tamaño, formato y campos obligatorios.
5. El API Gateway envía la solicitud al Material Service.
6. El Material Service almacena el archivo en MinIO o registra el enlace externo.
7. El Material Service guarda metadatos y versión inicial en PostgreSQL.
8. El sistema asigna estado `PENDING_REVIEW` y confirma la recepción.

Flujos alternativos

A1: Archivo supera el límite. Si el archivo local supera 15 MB, el sistema rechaza la subida y puede sugerir usar enlace externo.

A2: Clasificación incompleta. Si falta información obligatoria, el sistema no permite enviar el material a revisión.

A3: Nueva versión. Si el usuario sube una actualización de un material existente, el sistema crea una nueva `MaterialVersion` sin destruir necesariamente la anterior.

5.3. UC-03 Moderar y verificar material

Actor principal: Moderador o Administrador.

Precondiciones: Existe material en estado `PENDING_REVIEW` o reportes pendientes de revisión.

Postcondiciones: El material queda aprobado, rechazado, publicado, verificado, retirado o devuelto a revisión según corresponda.

Flujo principal

1. El moderador accede al panel administrativo.
2. El sistema muestra materiales pendientes y reportes abiertos.
3. El moderador revisa metadatos, archivo, pertinencia académica y eventuales reportes.
4. El moderador decide aprobar, rechazar, verificar, retirar o mantener el material.
5. El Quality Service registra la decisión y actualiza el estado correspondiente.
6. El sistema actualiza la visibilidad del material en búsquedas normales.

Flujos alternativos

A1: Material no pertinente. Si el contenido no corresponde a la clasificación declarada, el moderador lo rechaza o solicita corrección.

A2: Reporte confirmado. Si un reporte se confirma, el sistema puede disminuir la visibilidad, retirar el material o devolverlo a revisión.

5.4. UC-04 Buscar y filtrar material

Actor principal: Estudiante registrado.

Precondiciones: El usuario está autenticado y existen materiales publicados.

Postcondiciones: El usuario obtiene resultados ordenados por relevancia y señales de calidad.

Flujo principal

1. El usuario ingresa texto de búsqueda o selecciona filtros.
2. El sistema permite filtrar por carrera, asignatura, profesor, año, tipo de material y valoración.
3. El API Gateway deriva la consulta al Search Service.
4. El Search Service consulta los datos disponibles en PostgreSQL.
5. El sistema ordena resultados considerando relevancia, valoración, verificación, actualidad, reputación y reportes.
6. El usuario visualiza la lista de materiales y selecciona un resultado.

Flujos alternativos

A1: Sin resultados. Si no existen coincidencias, el sistema informa que no se encontraron materiales para los filtros aplicados.

A2: Material reportado. Si un material posee reportes relevantes, el sistema puede disminuir su posición o mostrar advertencias definidas por la plataforma.

5.5. UC-05 Previsualizar, descargar y guardar favorito

Actor principal: Estudiante registrado.

Precondiciones: El usuario está autenticado y el material está publicado.

Postcondiciones: El usuario visualiza, descarga o guarda el material como favorito.

Flujo principal

1. El usuario abre el detalle de un material.

2. El sistema muestra metadatos, valoración, estado de verificación y versiones disponibles cuando corresponda.
3. El usuario selecciona previsualizar, descargar o guardar favorito.
4. El API Gateway solicita autorización al backend.
5. El Material Service valida acceso y entrega el recurso autorizado.
6. Si el usuario marca favorito, el Quality Service registra la relación usuario-material.

Flujos alternativos

A1: Material no disponible. Si el material fue retirado o rechazado, el sistema no permite su descarga mediante búsquedas normales.

A2: Favorito duplicado. Si el material ya está en favoritos, el sistema evita duplicar la relación.

5.6. UC-06 Evaluar o reportar material

Actor principal: Estudiante registrado.

Precondiciones: El usuario está autenticado y el material está publicado.

Postcondiciones: La evaluación o reporte queda registrado y puede afectar reputación, visibilidad o moderación posterior.

Flujo principal

1. El usuario abre el detalle de un material.
2. El usuario selecciona calificar o reportar.
3. Si califica, el sistema registra una evaluación de 1 a 5 estrellas.
4. Si reporta, el sistema solicita tipo de reporte y descripción cuando corresponda.
5. El Quality Service registra la acción.
6. El backend recalcula señales de calidad, reputación o posicionamiento cuando corresponda.

Flujos alternativos

A1: Evaluación existente. Si el usuario ya tiene una evaluación activa para el material, el sistema actualiza la evaluación existente en lugar de crear otra.

A2: Reporte inválido. Si el reporte no contiene información mínima, el sistema solicita completar los datos.

6. Criterios de aceptación (extracto)

- Un usuario solo puede registrarse inicialmente con correo institucional `@uct.cl` o `@alu.uct.cl`. (RF-01)
- Un material nuevo debe quedar en estado `PENDING_REVIEW` y no aparecer como publicado hasta ser aprobado. (RF-03, RB-06)
- Todo material debe quedar clasificado por universidad, carrera, asignatura, profesor cuando corresponda, año y tipo de material. (RF-02, RB-05)
- Las consultas habituales de búsqueda deben responder aproximadamente en 2 segundos o menos bajo condiciones normales del entorno de prueba. (RNF-02)
- Web y Mobile deben producir resultados equivalentes al ejecutar una misma operación de negocio, ya que ambos consumen la misma API pública. (RNF-01, RNF-09)
- Un reporte confirmado no debe eliminar automáticamente el material sin revisión; debe pasar por el flujo definido por moderación. (RF-07, RB-09, RB-10)
- Una evaluación debe usar escala de 1 a 5 estrellas y debe existir como máximo una evaluación activa por usuario y material. (RF-06)
- Un usuario no debe poder duplicar el mismo material en favoritos. (RF-08)
- Los archivos deben almacenarse en MinIO o como enlaces externos seguros; PostgreSQL debe conservar metadatos y no el archivo completo. (RNF-04)
- La API pública debe estar documentada con OpenAPI/Swagger antes de la integración completa con Mobile. (RNF-06)

7. Trazabilidad (extracto)

Caso de uso	Requisito funcional	Regla	RNF
UC-01	RF-01	RB-01	RNF-01, RNF-04, RNF-09
UC-02	RF-02, RF-09	RB-05, RB-06, RB-15	RNF-04, RNF-05, RNF-08

UC-03	RF-03, RF-10, RF-12	RB-04, RB-06, RB-07, RB-08, RB-10, RB-14	RNF-07
UC-04	RF-04, RF-11	RB-07, RB-08, RB-11, RB-13	RNF-02, RNF-08
UC-05	RF-05, RF-08	RB-02, RB-08	RNF-04, RNF-09
UC-06	RF-06, RF-07	RB-03, RB-09, RB-10, RB-12, RB-13	RNF-07, RNF-10

8. Anexos

Anexo A: Ficha del proyecto

Campo	Detalle
Nombre del proyecto	ApuntesUCT - Ecosistema Colaborativo de Material Académico Validado
Institución objetivo	Universidad Católica de Temuco
Tipo de sistema	Plataforma web y móvil con backend de microservicios
Objetivo principal	Centralizar, validar y distribuir material académico contextualizado para estudiantes UCT
Duración estimada	Cuatro sprints para el MVP

Anexo B: Integrantes del proyecto

El proyecto está compuesto por 10 integrantes, separados en los equipos INT2 e INT4.

Equipo INT2

N	Nombre completo	Correo electrónico
1	Gabriel Esteban Gutierrez Yañez	ggutierrez2024@alu.uct.cl

2	Ailyn Alejandra Melillán Bustos	amelillan2025@alu.uct.cl
3	Martina Alejandra Iturrieta Liempi	miturrieta2025@alu.uct.cl
4	Raúl Alejandro Rodríguez Flores	rrodriguez2025@alu.uct.cl
5	David Ignacio Villegas Araneda	dvillegas2025@alu.uct.cl
6	Antonio Ignacio Lara Fuentealba	alara2024@alu.uct.cl

Equipo INT4

N	Nombre completo	Correo electrónico
1	Marcelo Fabian Santana Astorga	marcelo.santana2023@alu.uct.cl
2	Eduardo Javier Escares Ortiz	eescares2023@alu.uct.cl
3	Yaninna Rossio Álvarez Álvarez	yalvarez2023@alu.uct.cl
4	Nelson Javier Quiñinao Isla	nquininao2023@alu.uct.cl

Anexo C: Plan de actividades por sprint

Fase 1: Infraestructura y Arquitectura Base.

Configuración de Docker Compose, PostgreSQL y MinIO. Construcción inicial del API Gateway, arquitectura de microservicios con NestJS, integración de Prisma ORM y bases de Auth Service y Catalog Service. Generación del contrato OpenAPI inicial.

Fase 2: Gestión de Archivos y Descubrimiento.

Desarrollo del Material Service, subida multiformato, descarga de documentos, versionado básico, catálogo académico completo y búsqueda inicial.

Fase 3: Moderación y Ecosistema Colaborativo.

Desarrollo del Quality Service, evaluaciones, favoritos, reportes, moderación, verificación, reputación y primeras reglas de posicionamiento.

Fase 4: Refinamiento, Calidad y Despliegue.

Integración completa Web/Mobile/backend, refinamiento de filtros, pruebas automatizadas, seguridad, rendimiento, documentación y despliegue de demostración del MVP.

Anexo D: Resultados esperados

El proyecto ApuntesUCT debe entregar un MVP operativo compuesto por backend centralizado, cliente web responsivo y aplicación móvil nativa. Se espera contar con una base de datos estructurada, almacenamiento seguro de documentos, búsqueda contextual, previsualización, descarga, moderación, validación comunitaria, reputación y ranking de resultados.

El efecto esperado en la comunidad estudiantil es reducir el tiempo de recuperación de información útil, fomentar aprendizaje colaborativo y disminuir el uso de material obsoleto o no validado. Con la plataforma, los estudiantes podrán buscar apuntes según su contexto académico real y confiar en señales visibles de calidad, vigencia y reputación.

Anexo E: Diagramas requeridos del proyecto

- Diagrama de casos de uso UML - Auth: `diagramas/diagrama_casos_auth.png`.
- Diagrama de casos de uso UML - Search/Catalog: `diagramas/diagrama_casos_search.png`.
- Diagrama de casos de uso UML - Material/Quality: `diagramas/diagrama_casos_material_qu`.
- Diagrama de componentes UML: `diagramas/diagrama_componentes.png`.
- Diagrama de modelo entidad-relación (MER): `diagramas/diagrama_mer.png`.
- Diagrama de arquitectura de software: `diagramas/diagrama_arquitectura.png`.